scope_render.py — guide

Revision: 1 **Last modified:** 2026-09-26T13:36:00Z

Source: `constitution/scripts/codegraph/scope_render.py` (446 lines). Tests: `constitution/scripts/codegraph/tests/run_scope_tests.sh` (cases T01-T17, mutants M0-M15; the `§1.1-anchor:` markers in the source are the mutation targets) and `tests/t_scope_fixture.py` (the fixture builder). Citations below are `scope_render.py:<line>` unless another file is named.

Overview

Renders a project's CodeGraph index scope from ONE declarative file, `scope.yaml` (schema and template: `scope.example.yaml`), plus the project's `.gitmodules` (header, lines 3-28). It produces, deterministically:

1. `<root>/codegraph.json`, the only config file CodeGraph >= 1.6 reads (header line 9). Its `exclude` list is the root of every THIRD-PARTY submodule, then the baseline must-exclude classes, then project and pathological excludes (lines 309-317); an optional `include` list (from `include_patterns`, lines 337-339); and an inert `_generated` provenance key with the sha256 of the scope file and of every `.gitmodules` read (lines 323-336).
2. A root `.gitignore` negation block delimited by `# BEGIN helix-codegraph-scope` / `# END helix-codegraph-scope`, containing `!<pattern>` for each `reinclude_negations` entry (lines 341-343). Only a root `.gitignore` negation can re-include owned source under a built-in-skipped parent; the config's `include` cannot (header lines 23-27).

Own-organisation submodule paths never appear in `exclude`; a guard in the renderer refuses if one would (lines 319-322). `render` is a pure function of the scope file bytes and the `.gitmodules` bytes (docstring, line 302).

The judgement side (does the engine actually enumerate what this scope promises?) is `codegraph_scope_guard.py`; this tool only renders and checks freshness.

Prerequisites

- `python3` with PyYAML (imported lazily, lines 74-77; missing PyYAML is a fail-closed exit 3) and `git` (used to parse `.gitmodules`, line 145).
- A scope file with the required keys `schema_version` (must be 1), `own_orgs` (non-empty `hosts` and `orgs`) and `baseline_excludes` (non-empty `build_outputs`, `caches`, `secrets`, `qa_corpora`) (lines 58-59, 88-105).
- `<root>/.gitmodules` is optional; without it there are simply no submodule roots (line 180).

Usage

```
scope_render.py --scope <scope.yaml> --root <project root> [--write | --check]
                [--out-config FILE] [--out-gitignore-block FILE]
```

Argument	Meaning	Source
`--scope FILE`	Scope file. Required	line 404
`--root DIR`	Project root. Required; must be a directory (else exit 3)	lines 405, 415-418
`--write`	Write `<root>/<config filename>` and splice the block into `<root>/.gitignore`	lines 407, 430-434
`--check`	Do not write; report drift between the disk and a fresh render; exit 1 on drift	lines 408, 435-438
`--write` and `--check`	Mutually exclusive (argparse error, exit 3)	line 406
`--out-config FILE`	Also write the rendered config to FILE (atomic)	lines 409, 426-427
`--out-git-ignore-block FILE`	Also write the rendered block to FILE (atomic)	lines 410, 428-429
(no mode flag and no `--out-*`)	Print the rendered config JSON on stdout	lines 439-440

The config file name is `runner.config_filename` from the scope, default `codegraph.json` (line 348).

Examples

Run against a generated throwaway fixture repository (`tests/t_scope_fixture.py` : real git submodules, one own-org submodule with a nested third-party submodule, one top-level third-party submodule) in a scratch directory. The repository's real tree and index were never used.

#	Command	Status
1	`scope_render.py --help`	VERIFIED (ran; saw argparse usage, exit 0)
2	`scope_render.py` (no arguments)	VERIFIED (ran; saw `error: the following arguments are required: --scope, --root`, exit 3)
3	`scope_render.py --scope <fx>/code-graph.scope.yaml --root <fx>`	VERIFIED (ran; stdout was the JSON config with `_generated`, `exclude` = `/own_sub/` `vendored_tp/`, `/tp_top/`, `/out/`, `**/__pycache__/`, `.env`, `*.env`, `*secret*`, `!*secret*/`, `**/secrets/`, `**/qa-results/`, `/rubbish/`, and `include` = `/app/vend/gen/`; stderr one `SCOPE_RENDER {...}` line; the fixture's `git status` was unchanged)
4	Same with `--check` before anything was written	VERIFIED (ran; saw `"drift": ["config_missing", "block_missing"]`, exit 1)
5	Same with `--out-config <f> --out-git-ignore-block <g>`	VERIFIED (ran; `<g>` held the three-line block `# BEGIN ... !/build/ ... # END ...`; nothing was written to the root)
6	Same with `--write`	VERIFIED (ran; exit 0; `git status` then showed `codegraph.json` untracked and `.gitignore` modified; the block was the last content of `.gitignore`)
7	`--check` after `--write`	VERIFIED (ran; saw `"drift": []`, exit 0)
8	`--write` a second time	VERIFIED (ran; `md5sum` of `codegraph.json` and `.gitignore` identical before and after)
9	Append a line after the block, then `--check`	VERIFIED (ran; saw `"drift": ["block_not_last"]`, exit 1)

#	Command	Status
10	`--write --check` together	VERIFIED (ran; argparse `not allowed with argument --write`, exit 3)
11	Missing scope file / root not a directory / unparsable YAML	VERIFIED (ran; `FAIL-CLOSED: cannot read scope file ...`, `root is not a directory: ...`, `FAIL-CLOSED: unparsable scope file ...`; exit 3 for all three)
12	Scope with an extra top-level key `bogus_key`	VERIFIED (ran; `FAIL-CLOSED: unknown top-level scope key(s) ... bogus_key`)
13	Scope with the `caches` baseline class removed	VERIFIED (ran; `FAIL-CLOSED: baseline class caches missing or empty (§11.4.78: a class may not be removed)`)
14	Scope whose `secrets` list has `**/secrets/` before `!*secret*/`	VERIFIED (ran; `FAIL-CLOSED: exclusion '**/secrets/' (index 4) is voided by the later negation '!*secret*/' (index 5) ... write '**/secrets/' AFTER '!*secret*/'`)
15	Scope with `rein-clude_negations: ["!/build/"]`	VERIFIED (ran; `FAIL-CLOSED: rein-clude_negations entries are written WITHOUT the leading '!': !/build/`)
16	Unclassifiable gitlink (relative or missing URL, no override)	NOT RUN (covered by the source at lines 190-193, 202-203; exercised by the test suite, not by hand)

Inputs

- The scope file, parsed with `yaml.safe_load` and validated by `load_scope` (lines 73-119):
  - required keys, and every top-level key must be in `KNOWN_KEYS` (lines 59-62, 91-94): an unknown key is refused "because a silently ignored key would silently drop a rule";

    - `own_orgs.hosts` and `own_orgs.orgs` non-empty; the four required baseline classes non-empty (a class may be added but not removed, lines 58, 103-105);
    - `project_excludes`, `pathological_excludes`, `reinclude_negations`, `include_patterns`: lists of non-empty strings; negations and includes must be written without a leading `!` (lines 106-115);
    - `submodule_class_overrides` values must be `own` or `third_party` (lines 116-118). The guard-only keys `forbidden_classes`, `enumeration_controls`, `accepted_count`, `tolerance_pct` and `runner` are accepted here and ignored by the renderer (they are in `KNOWN_KEYS`, lines 60-62).
- `<root>/.gitmodules` and, recursively, the `.gitmodules` of every checked-out OWN-org submodule (lines 177-199). Third-party subtrees are excluded whole and not descended into (line 198-199).

Outputs

- stdout: the rendered config only when neither `--write`, `--check`, nor an `--out-*` option is given (lines 439-440).
- stderr: one line `SCOPE_RENDER <json>` (sorted keys) with the counts `own_org_roots`, `third_party_roots`, `exclude_patterns`, `include_patterns`, `negations`, `order_lint_unprobed`, `gitmodules_files`, `config_filename`, plus `drift` with `--check` (lines 344-348, 437, 441). Failures print `scope_render: FAIL-CLOSED: <reason>` (line 422).
- Files: with `--write`, `<root>/<config filename>` and `<root>/.gitignore`; with `--out-*`, the named files. All written atomically through `<path>.scope_render.tmp` then `os.replace` (lines 395-399).

Exit codes

Code	Meaning	Source
0	Rendered / written / `--check` found no drift / `--help`	lines 414, 442
1	`--check` found drift	line 438
3	Fail-closed: unparsable or invalid scope, missing baseline class, order-lint violation, unclassifiable gitlink, unreadable `.gitmodules`, root not a directory, usage error (including argparse errors)	lines 413-414, 417-418, 421-423

Ownership rule

A submodule is OWN when its URL host is in `own_orgs.hosts` and its org is in `own_orgs.orgs` (both compared lower-cased), otherwise THIRD-PARTY (lines 172-173, 190-194). A path in `submodule_class_overrides` wins over the URL (lines 174, 188). `url_host_org` understands `git@host:org/repo`, `ssh://[user@]host[:port]/org/repo` and `http(s)/git://[user@]host[:port]/org/repo`; a URL that is empty, starts with `./`, `../`, `/` or `file:`, or whose host has no dot, has no owner and must be classified by an override or the renderer refuses (lines 130-139, 202-203). Nested submodules of an own-org submodule are classified by their own URL (line 197).

Edge cases

- **Exclusion order is linted (fail closed).** The engine's matcher lets the LAST matching pattern decide, so a later `!X` can silently re-include something an earlier exclusion covered. `check_exclusion_order` synthesises a probe path for each earlier exclusion and refuses (exit 3) when a later negation (other than the one with the same pattern body) matches it and no still-later exclusion re-asserts it (lines 261-289; example 14). Exclusions whose pattern cannot be probed are counted in `order_lint_unprobed` and not checked (lines 279-282, 346). The lint sees only the `exclude` list of one render; it does not look at the negations in the `.gitignore` block.

- **The block is always moved to the end of `.gitignore`.** `splice_gitignore` removes every existing block and appends the new one after the remaining content (lines 355-358). `--check` reports one of `block_missing`, `block_duplicated`, `block_differs`, `block_not_last` (lines 361-373; example 9). A missing `.gitignore` reads as empty (lines 386-388).
- **`--write` overwrites `<root>/<config filename>` unconditionally** (line 431); it does not merge with a hand-edited config. Every rendered config carries `_generated.do_not_edit`.
- **`--check` still honours `--out-*`.** Output files are written before the mode is evaluated (lines 426-429).
- **Provenance is content-addressed.** `_generated.scope_sha256` and `gitmodules_sha256` change when the scope file or any `.gitmodules` read changes, so any input edit makes the on-disk config differ from a fresh render (lines 306-308, 329-330).
- **Deterministic output.** JSON is written with sorted keys and the exclude list is de-duplicated preserving first occurrence (lines 292-298, 340).
- **A leftover `<path>.scope_render.tmp` can remain** only if the process dies between the write and the replace (lines 396-399). UNCONFIRMED: not provoked.

Internal behaviour

1. `load_scope` validates the scope (lines 73-119).
2. `derive_submodules` walks `.gitmodules` recursively and classifies each gitlink (lines 169-205); `.gitmodules` is parsed with `git config -f <file> --null --get-regexp` (lines 142-166); a return code other than 0/1 is an unparsable file (line 149).
3. Build `exclude` = third-party roots (`/<path>/`), baseline classes (required ones first in a fixed order, then any added classes in written order), project excludes, pathological excludes; de-duplicate; run the order lint (lines 309-318).
4. Build the config dict and the negation block (lines 323-343).
5. Depending on the mode, write, compare with the disk, or print (lines 424-441).

Related

File	Relation
code-graph_scope_guard	Imports this module (`gitignore_to_regex` , `check_on_disk` , `render` , `splice_gitignore` , `atomic_write` , `load_scope`) and judges the resulting scope against the engine's real enumeration
scope_enumerate	The bridge into the runner's own file discovery
`scope.example.yaml`	Schema and template for the scope file
`scope_baseline.txt`	Must-exclude classes data file for the `code-graph_safe.sh` scope-proof stage (its header, lines 1-3); not read by this renderer
owed_report	Reports whether this tool has tests and a guide

Last verified

2026-09-26 (source lines read on that date; examples run on a scratch fixture, no real index or repository tree touched). Full scope test suite run in a scratch `TMPDIR` : T01-T17 PASS, M0-M15 PASS (18 results), determinism verdict PASS (one iteration).

Findings: none. The in-source header block (lines 2-47) matches the code, including its exit-code list (0/1/3) and side effects.